

NDAT

Arayüz Ayırıştırma İlkesi (Interface Segregation Principle - ISP),

- "İstemciler (client), kullanmadıkları metotlara bağımlı olmaya zorlanmamalıdır."

Neden Önemlidir?

- **Esneklik:** Bir değişiklik yapıldığında sadece ilgili sınıflar etkilenir.
- **Temiz Kod:** Sınıflar sadece gerçekten yaptıkları işleri içeren metotlara sahip olur.
- **Bakım Kolaylığı:** Kodun anlaşılabilirliği artar.

Örnek

```
public interface ISmartDevice {
    void Print();
    void Scan();
    void Fax();
}

// Gelişmiş yazıcı her şeyi yapar, sorun yok.
public class HighEndPrinter : ISmartDevice {
    public void Print() { /* Yazdırıyor... */ }
    public void Scan() { /* Tarıyor... */ }
    public void Fax() { /* Faks çekiyor... */ }
}

// AMA sadece yazdırma özelliği olan eski bir yazıcı eklersek:
public class OldSchoolPrinter : ISmartDevice {
    public void Print() { /* Yazdırıyor... */ }

    public void Scan() {
        throw new NotImplementedException(); // HATA: Bu cihaz tarayamaz!
    }

    public void Fax() {
        throw new NotImplementedException(); // HATA: Bu cihaz faks çekemez!
    }
}
```

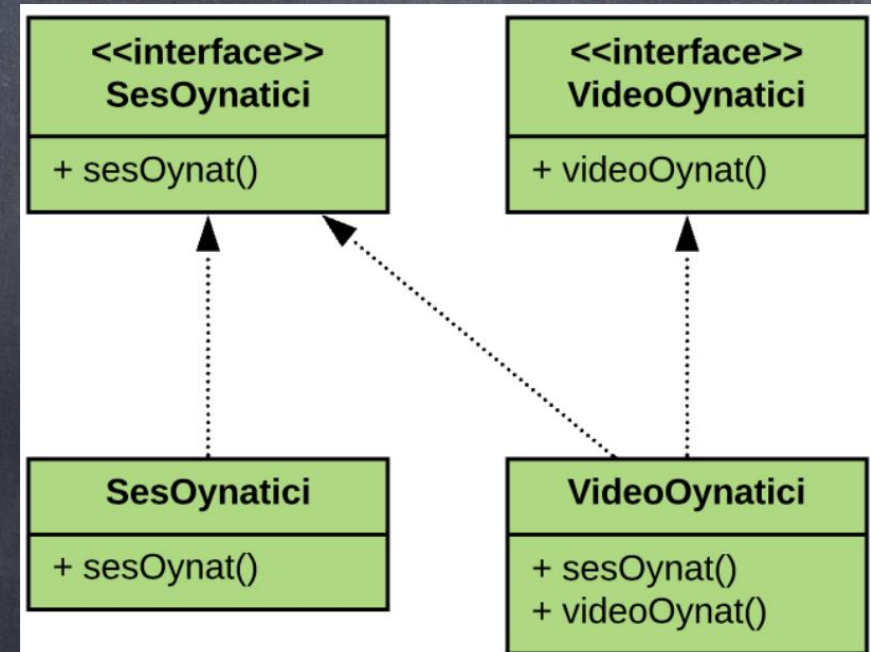
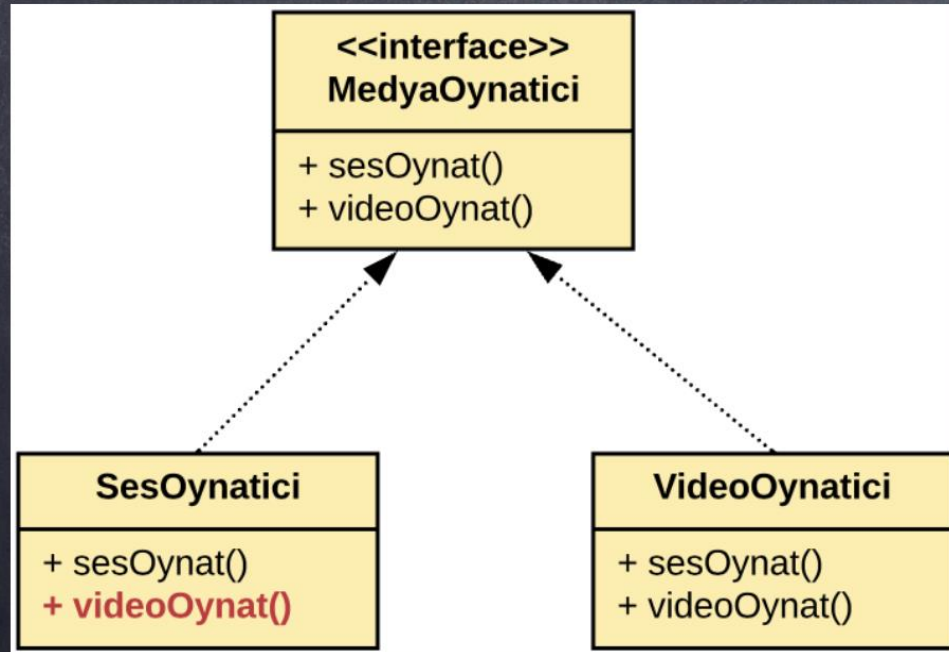
```
public interface IPrinter {
    void Print();
}

public interface IScanner {
    void Scan();
}

public interface IFax {
    void Fax();
}

// Eski yazıcı sadece yazdırır
public class OldSchoolPrinter : IPrinter {
    public void Print() {
        Console.WriteLine("Yalnızca çıktı alınıyor.");
    }
}

// Yeni nesil yazıcı hepsini birden uygulayabilir (Multiple Inheritance)
public class HighEndPrinter : IPrinter, IScanner, IFax {
    public void Print() { /* ... */ }
    public void Scan() { /* ... */ }
    public void Fax() { /* ... */ }
}
```



Bağımlılığın Tersine Çevrilmesi İlkesi (Dependency Inversion Principle)

DIP temel olarak iki ana kurala dayanır:

- **Yüksek seviyeli modüller, düşük seviyeli modüllere bağımlı olmamalıdır.** Her ikisi de soyutlamalara (abstraction) bağımlı olmalıdır.
- **Soyutlamalar detaylara bağımlı olmamalıdır. Detaylar soyutlamalara bağımlı olmalıdır.**

Örnek

```
// Düşük Seviyeli Sınıf (Detay)
public class SMS {
    public void SendSMS(string message) {
    }

    // Yüksek Seviyeli Sınıf (İş mantığı)
    public class NotificationManager {
        private SMS _sms = new SMS(); // SIKI

        public void Send(string message) {
            _sms.SendSMS(message);
        }
    }
}
```

```
public interface IMessageService {
    void SendMessage(string message);
}

// Düşük Seviyeli Sınıflar (Detaylar artık interface'e bağımlı)
public class SMS : IMessageService {
    public void SendMessage(string message) { /* SMS kodları */ }
}

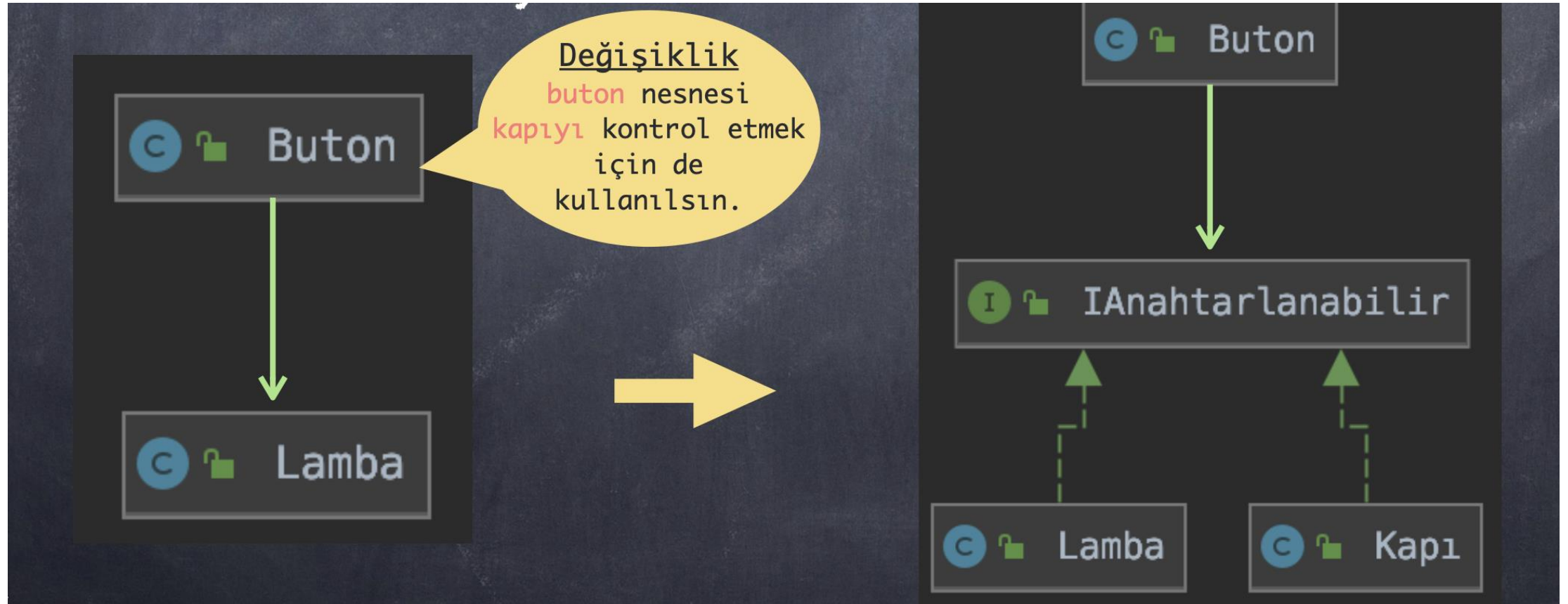
public class Email : IMessageService {
    public void SendMessage(string message) { /* Email kodları */ }
}

// Yüksek Seviyeli Sınıf (Artık sadece interface'e bağımlı)
public class NotificationManager {
    private readonly IMessageService _messageService;

    // Dependency Injection (Bağımlılık Enjeksiyonu)
    public NotificationManager(IMessageService service) {
        _messageService = service;
    }

    public void Send(string message) {
        _messageService.SendMessage(message);
    }
}
```

DIP



Tasarım Desenleri

- Nesne yönelimli programlamada sıkça karşılaşılan problemlere karşı geliştirilmiş "standart çözüm şablonlarıdır."
- **1. Oluşturucu (Creational) Desenler**
- Nesnelerin nasıl oluşturulduğuyla ilgilenirler. "New" anahtar sözcüğünü doğrudan kullanmak yerine, nesne yaratma sürecini soyutlarlar. Örnek: Singleton, Factory Method
- **2. Yapısal (Structural) Desenler**
- Sınıf ve nesnelerin bir araya gelerek daha büyük yapıları nasıl oluşturacağıyla ilgilenirler. Örnek: **Adapter (Adaptör), Decorator**

Tasarım Desenleri

- **3. Davranışsal (Behavioral) Desenler**
- Nesnelerin birbirleriyle nasıl etkileşime gireceğini ve sorumlulukların nasıl dağıtılacağını belirlerler. Örnek: **Observer (Gözlemci), Strategy**

Tasarım Desenleri

- Programlama dillerinden bağımsızdır.
- Yazılım geliştiricilerin tasarımlarla ilgili tartışma yapmasını kolaylaştırır.

Tasarım desenleri dört bölümden oluşur

- Desenin adı
- problemin tanımı: desenin ne zaman/nerede kullanılabileceği
- çözüm: problemin nasıl çözüleceği (kullanılması gereken bileşenler, aralarındaki bağıntı vb.)
- sonuç: deseni uygulamanın sonuçları?

Creational (Nesne oluřturma)

Singleton Pattern

Factory Pattern

Abstract Factory Pattern

Builder Pattern

Prototype Pattern

Structural (Yapısal)

Adapter Pattern

Composite Pattern

Proxy Pattern

Flyweight Pattern

Facade Pattern

Bridge Pattern

Decorator Pattern

Behavioral (Davranıřsal)

Template Method Pattern

Mediator Pattern

Chain of Responsibility
Pattern

Observer Pattern

Strategy Pattern

Command Pattern

State Pattern

Visitor Pattern

Interpreter Pattern

Iterator Pattern

Singleton Tasarım Deseni

- **Singleton**, bir sınıftan uygulama ömrü boyunca **sadece bir tane nesne (instance)** oluşturulmasını garanti eden ve bu nesneye her yerden erişilmesini sağlayan bir tasarım desenidir.
- **Temel Kuralları:**
- **Private Constructor (Gizli Kurucu):** Sınıfın dışından new anahtar kelimesiyle yeni bir nesne oluşturulmasını engeller.
- **Static Instance (Statik Örnek):** Sınıfın kendi içinde, oluşturulan tek nesneyi tutan statik bir değişkeni vardır.
- **Static GetInstance (Erişim Metodu):** Her yerden bu tek nesneye erişim sağlayan, genellikle GetInstance() adında bir metod sunar.

Örnek

```
public class DatabaseConnection {  
    // 1. Tek nesneyi tutacak statik değişken  
    private static DatabaseConnection _instance;  
  
    // 2. Constructor private olmalı ki dışarıdan 'new' yapılamasın  
    private DatabaseConnection() {  
        Console.WriteLine("Veritabanına bağlanıldı.");  
    }  
  
    // 3. Nesneye erişim noktası  
    public static DatabaseConnection GetInstance() {  
        if (_instance == null) {  
            _instance = new DatabaseConnection();  
        }  
        return _instance;  
    }  
  
    public void ExecuteQuery(string query) {  
        Console.WriteLine($"Sorgu çalıştırılıyor: {query}");  
    }  
}
```

```
// DatabaseConnection db = new DatabaseConnection(); // HATA! Kurucu private.
```

```
DatabaseConnection db1 = DatabaseConnection.GetInstance();
```

```
DatabaseConnection db2 = DatabaseConnection.GetInstance();
```

```
// db1 ve db2 aslında bellekteki aynı nesneyi işaret eder.
```

Singleton Tasarım Deseni

Avantajlar

Kontrollü Erişim: Nesneye kimin, ne zaman erişeceği bellidir.

Kaynak Tasarrufu: Gereksiz bellek kullanımını önler.

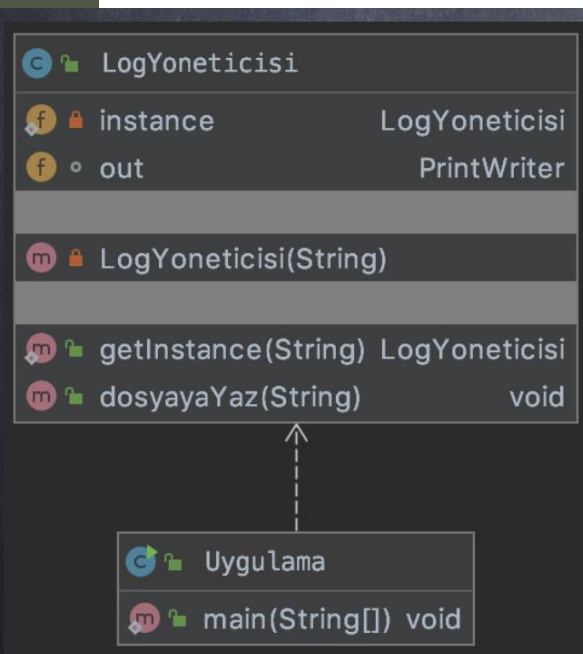
Global Erişim: Her yerden ulaşılabilir.

Dezavantajlar

Unit Test Zorluğu: Statik yapısı nedeniyle test edilmesi (mocking) zordur.

Multi-threading: Aynı anda iki thread nesne yaratmaya çalışırsa iki örnek oluşabilir (Thread-safety önlemi gerekir).

Gizli Bağımlılık: Kodun her yerinde kullanılması, sınıflar arası bağımlılığı (coupling) artırabilir.



```
public class LogYoneticisi {

    private static LogYoneticisi instance;
    PrintWriter out;

    private LogYoneticisi(String logDosyasi){
        try {
            out = new PrintWriter(new FileWriter(logDosyasi,true), true);
        } catch (IOException e) {e.printStackTrace();}
    }

    public static synchronized LogYoneticisi getInstance(String logDosyasi){
        if(instance==null)
            instance = new LogYoneticisi(logDosyasi);
        return instance;
    }

    public void dosyayaYaz(String mesaj) {
        out.println(LocalDate.now()+":"+mesaj);
    }
}

public class Uygulama {
    public static void main(String [] args){
        LogYoneticisi.getInstance("Log.txt").dosyayaYaz("[WARNING]:uyari mesajı 1");
    }
}
```

Observer Tasarım Deseni

- Nesneler arasında **bire-çok (one-to-many)** bir bağımlılık kurmak için kullanılır. Bir nesnenin durumu değiştiğinde, ona bağlı olan tüm diğer nesnelerin otomatik olarak haberdar edilmesini ve güncellenmesini sağlar.

- **Temel Mantık: Nasıl Çalışır?**

İki ana aktör vardır:

- **Subject (Konu/Yayıncı):** Takip edilen nesnedir. İçinde gözlemcilerin bir listesini tutar. Durumu değiştiğinde listedeki herkese haber verir.
- **Observer (Gözlemci/Abone):** Haber bekleyen nesnedir. Yayıncıdan bildirim geldiğinde kendi içinde bir işlem yapar.

```
// 1. Gözlemci arayüzü
public interface IObservable {
    void Update(string news);
}

// 2. Yayıncı (Subject) sınıfı
public class NewsAgency {
    private List<IObservable> _observers = new List<IObservable>();

    public void Subscribe(IObservable observer) {
        _observers.Add(observer);
    }

    public void Unsubscribe(IObservable observer) {
        _observers.Remove(observer);
    }

    public void Notify(string news) {
        foreach (var observer in _observers) {
            observer.Update(news);
        }
    }
}

// 3. Somut Gözlemciler (Concrete Observers)
public class EmailApp : IObservable {
    public void Update(string news) {
        Console.WriteLine($"E-posta gönderildi: {news}");
    }
}

public class MobileApp : IObservable {
    public void Update(string news) {
        Console.WriteLine($"Mobil bildirim gönderildi: {news}");
    }
}
}
```

```
NewsAgency agency = new NewsAgency();
```

```
IObservable user1 = new EmailApp();
```

```
IObservable user2 = new MobileApp();
```

```
agency.Subscribe(user1);
```

```
agency.Subscribe(user2);
```

```
// Haber yayınlandığında herkes haberdar olur
```

```
agency.Notify("Tasarım Desenleri sınavı ertelendi!");
```

Observer Tasarım Deseni

Avantajlar

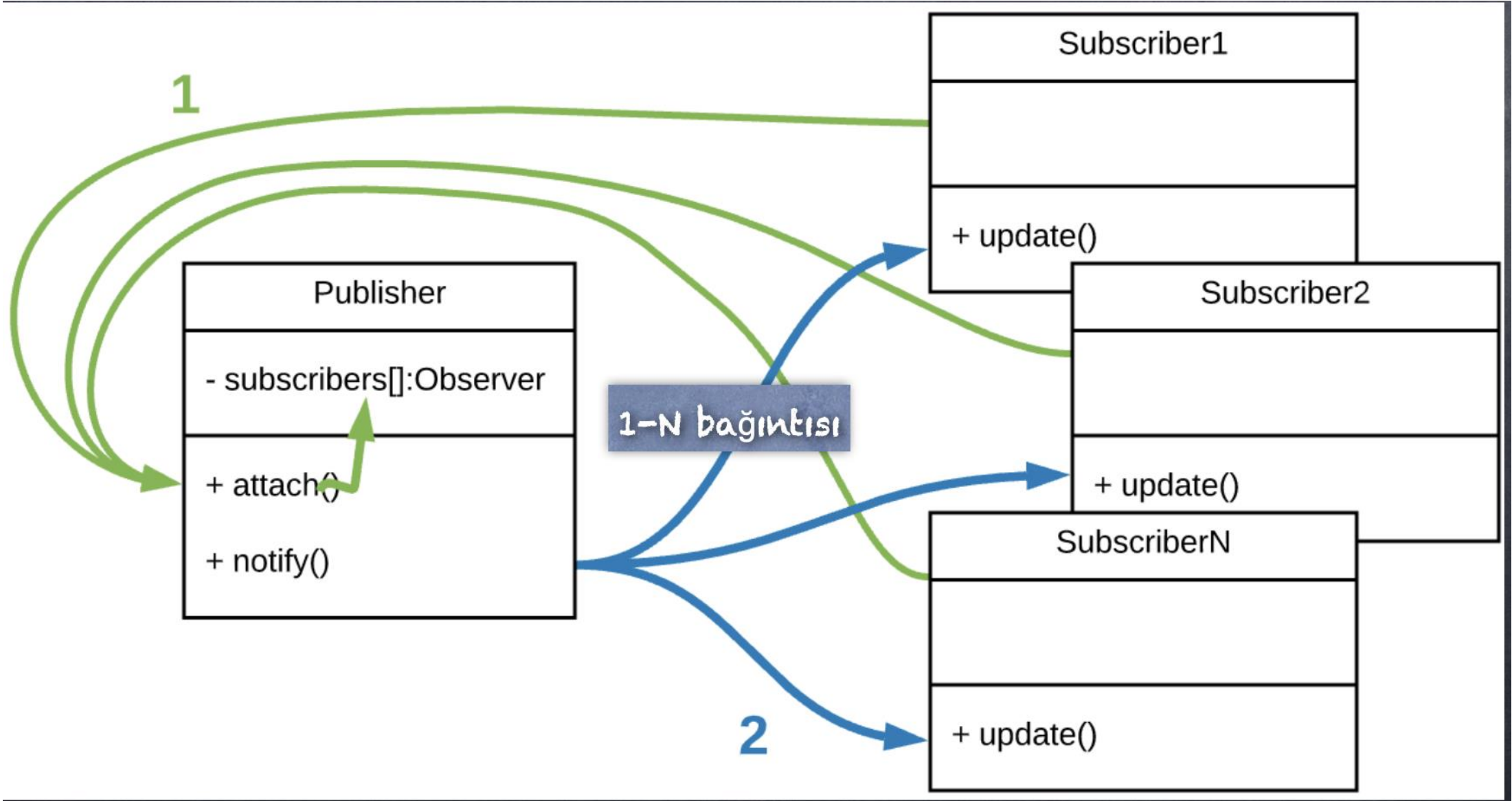
Gevşek Bağlılık (Loose Coupling): Yayıncı, gözlemcilerin kim olduğunu veya ne iş yaptığını bilmez.

Dinamik Abonelik: Çalışma anında (runtime) yeni aboneler eklenebilir veya çıkarılabilir.

Dezavantajlar

Sıralama Garantisi Yok: Bildirimlerin hangi sırayla gideceği genellikle garanti edilmez.

Bellek Sızıntısı: Eğer abonelikten çıkma (unsubscribe) unutulursa, nesneler bellekten silinmeyebilir.



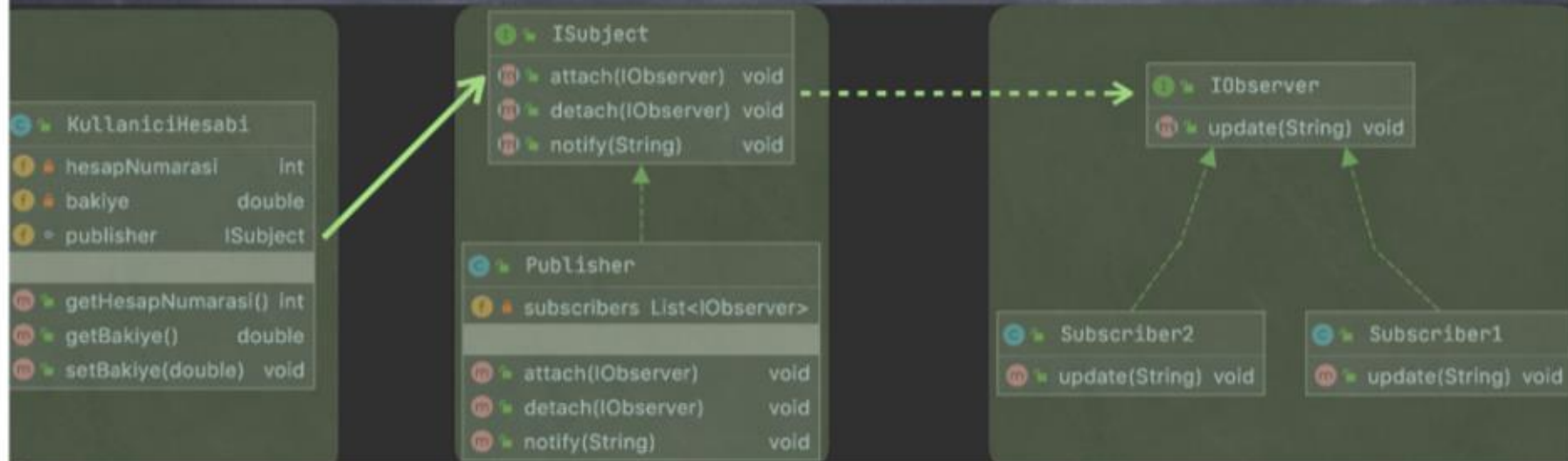
- * Subject (publisher- yayıncı): gözlemcilerin kaydedilmesi/çıkarılması, istemcideki değişikliklerin gözlemcilere bildirilmesi işlemlerinden sorumludur.
- * Observer (subscriber-abone): istemcideki olayların yansıtıldığı/gönderildiği nesne.
- * İstemci (kullaniciHesabi): olayın üretildiği nesne.

Örnek: Kullanıcının bakiyesi değiştiğinde, bu olayı abonelere bildiren uygulama.

istemci(event source)

subject (publisher)

observer (subscriber)



Adapter Tasarım Deseni

- Birbirleriyle uyumlu olmayan iki arayüzün (interface) birlikte çalışmasını sağlayan bir **yapısal (structural) tasarım desenidir**.

Adapter deseni genellikle dört ana parçadan oluşur:

- **Client (İstemci)**: Mevcut sistemi kullanan ve belirli bir standart arayüze göre çalışan sınıftır.
- **Target (Hedef Arayüz)**: Client'ın beklediği veya anladığı arayüzdür.
- **Adaptee (Uyarlanan)**: Sisteme dahil edilmek istenen ancak arayüzü mevcut yapıya uymayan eski veya üçüncü taraf sınıftır.
- **Adapter (Adaptör)**: Target arayüzünü uygular ve Adaptee'den gelen çağrıları Target'ın anlayacağı dile çevirir.

Json ve XML Entegrasyonu

1. Uyumsuz Sınıf (Adaptee)

C#

```
class XMLDataPoster {  
    public string GetXMLData() {  
        return "<data><value>100</value></data>";  
    }  
}
```

2. Beklenen Arayüz (Target)

C#

```
interface IJsonAnalyzer {  
    void Analyze(string jsonData);  
}
```

3. Adaptör Sınıfı (Adapter)

Adaptör, XML sınıfını alır ve içindeki veriyi JSON formatına çevirerek hedef arayüze sunar.

C#

```
class XmlToJsonAdapter : IJsonAnalyzer {  
    private XMLDataPoster _xmlPoster;  
  
    public XmlToJsonAdapter(XMLDataPoster xmlPoster) {  
        _xmlPoster = xmlPoster;  
    }  
  
    public void Analyze(string jsonData) {  
        // XML'i alıp JSON'a çevirme mantığı burada çalışır  
        string xml = _xmlPoster.GetXMLData();  
        string convertedJson = "{ \"value\": 100 }";  
  
        Console.WriteLine("Dönüştürülen veri analiz ediliyor: " + convertedJson);  
    }  
}
```

Avantajları

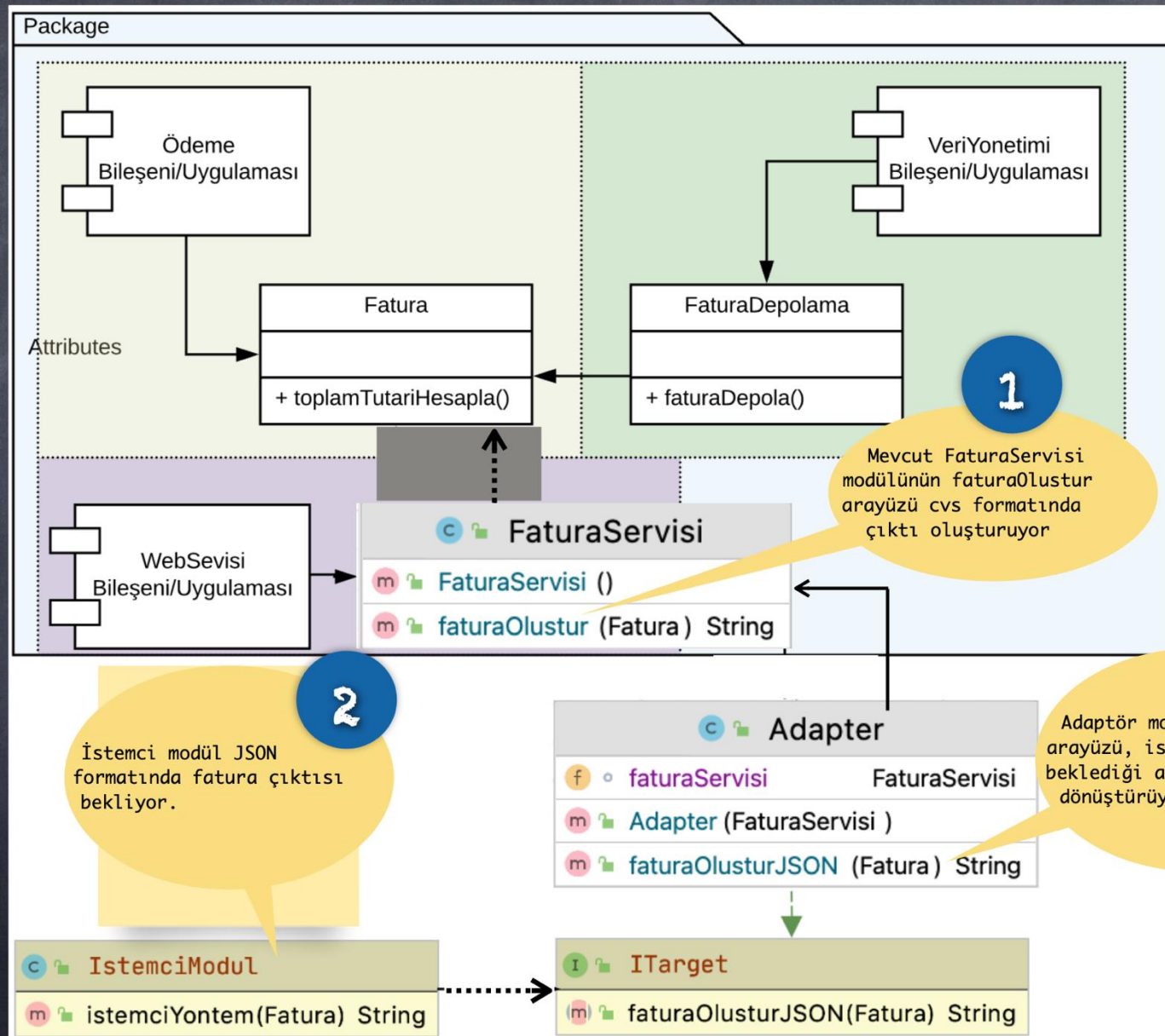
Sorumlulukların Ayrılması: Dönüşüm mantığı ana koddan ayrılır.

Esneklik: Mevcut kodu bozmadan yeni sınıflar eklenebilir (Open/Closed Principle).

Dezavantajları

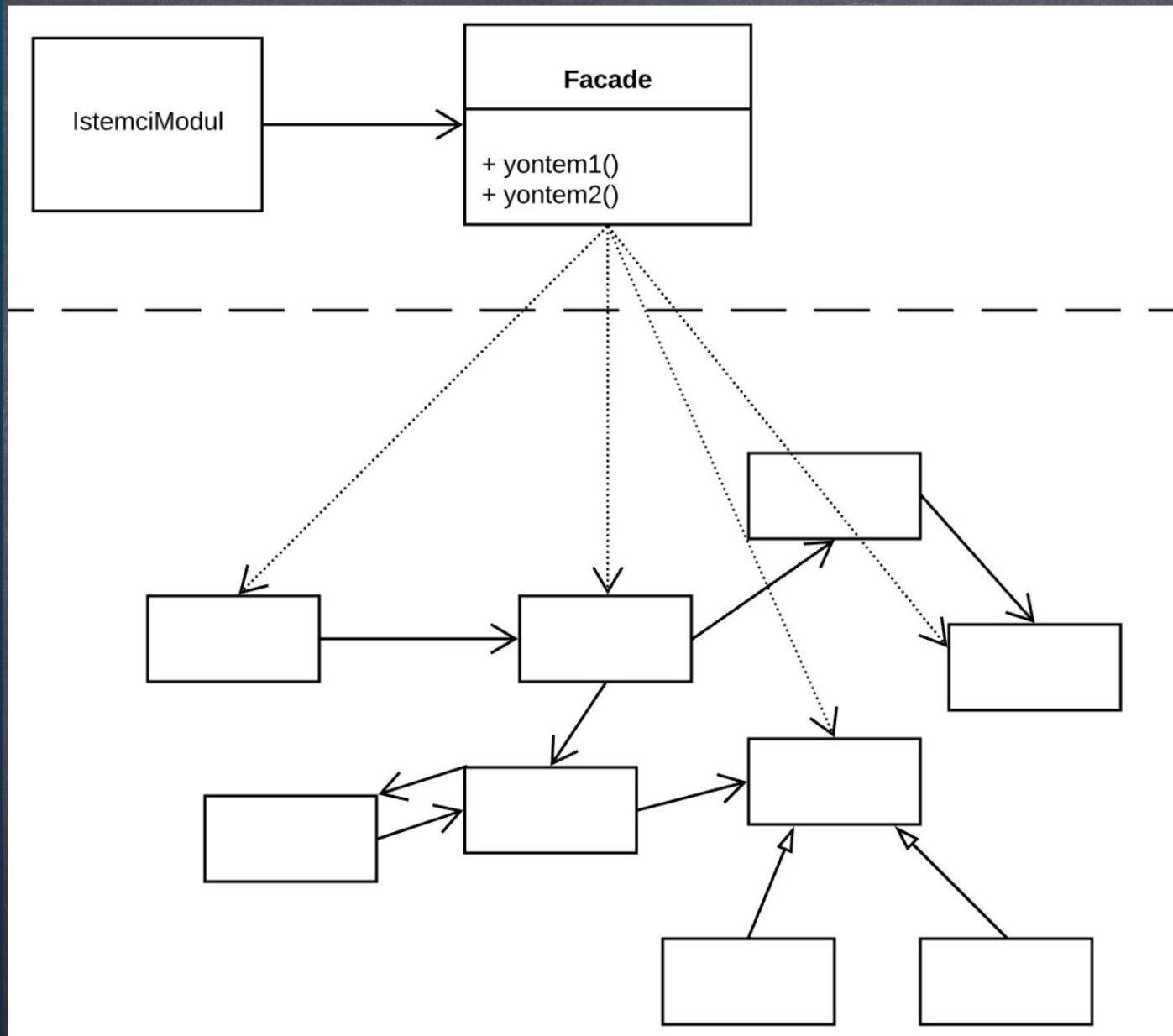
Karmaşıklık: Çok fazla yeni arayüz ve sınıf eklendiği için kodun genel karmaşıklığı artabilir.

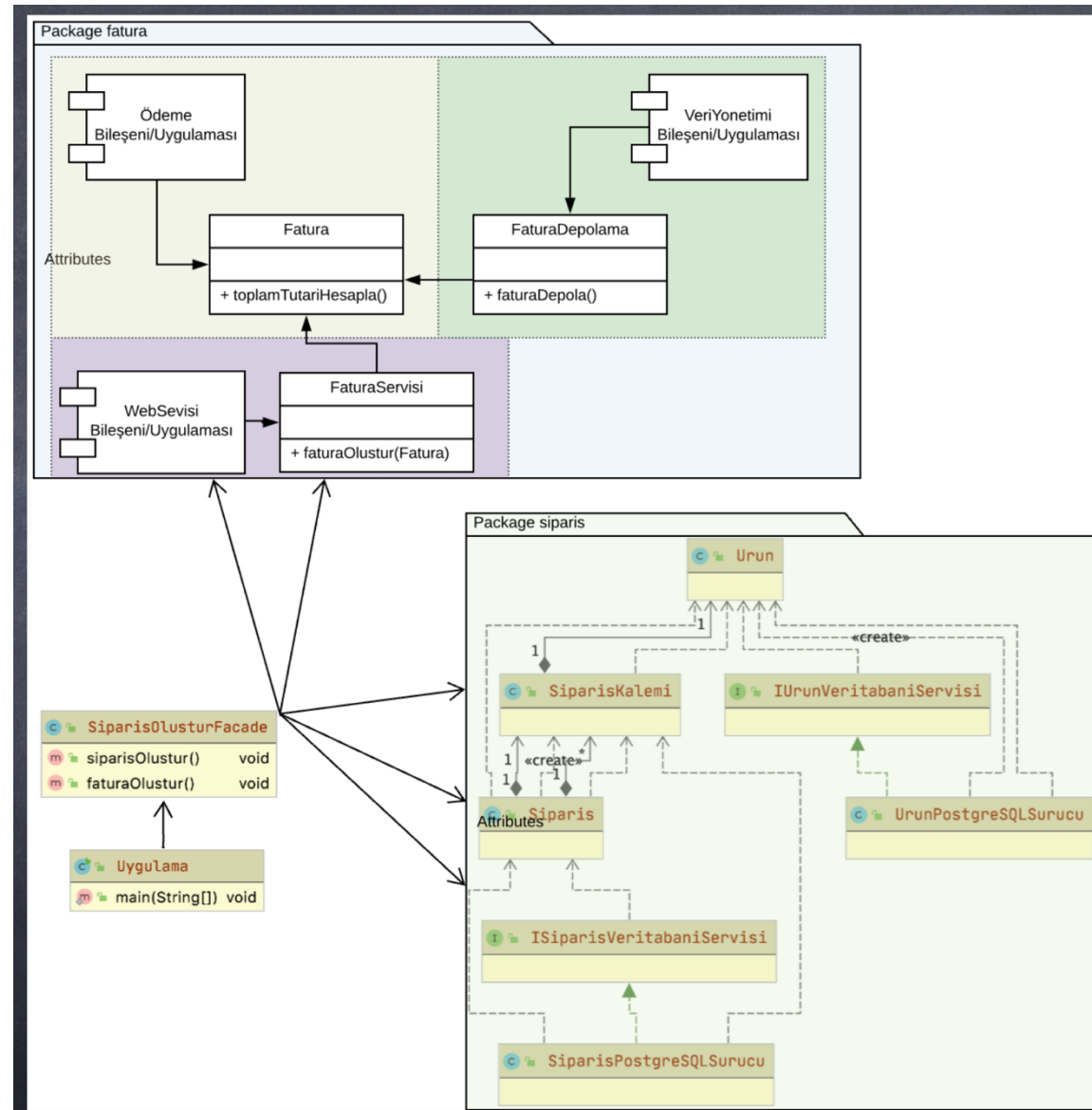
Performans: Arada bir çevirici katman olduğu için çok kritik sistemlerde küçük bir gecikme yaratabilir.



Facade Tasarım Deseni

- **Facade (Ön Yüz)** tasarım deseni, karmaşık bir alt sistemdeki sınıflar topluluğuna yönelik **basitleştirilmiş ve yüksek seviyeli bir arayüz** sağlar.
- Bir kütüphaneyi veya sistemi kullanmak için onlarca sınıfı başlatmanız, bağımlılıkları yönetmeniz ve metodları belirli bir sırayla çağırmanız gerekiyorsa, Facade deseni bu karmaşıklığı tek bir sınıfın içine paketler.
- **Alt Sistem Karmaşıklığını Gizlemek:** İstemci (Client) alt sistemin detaylarını bilmek zorunda kalmaz.
- **Gevşek Bağlılık (Loose Coupling):** İstemci sadece Facade'e bağlıdır, alt sistemdeki değişikliklerden etkilenmez.





Akıllı Ev Sinema Sistemi

- Bir film izlemek istediğinizde şu adımları yapmanız gerekir:
- Işıkları kıs.
- Projeksiyonu aç.
- Ses sistemini aç ve ses seviyesini ayarla.
- DVD oynatıcıyı başlat.

1. Karmaşık Alt Sistem Sınıfları

C#



```
class Lights { public void Dim() => Console.WriteLine("Işıklar kısıldı."); }  
class Projector { public void On() => Console.WriteLine("Projeksiyon açıldı."); }  
class SoundSystem { public void SetVolume(int level) => Console.WriteLine($"Ses s
```

İstemcinin bu sınıflarla tek tek uğraşması yerine, bir "HomeTheaterFacade" oluştururuz.

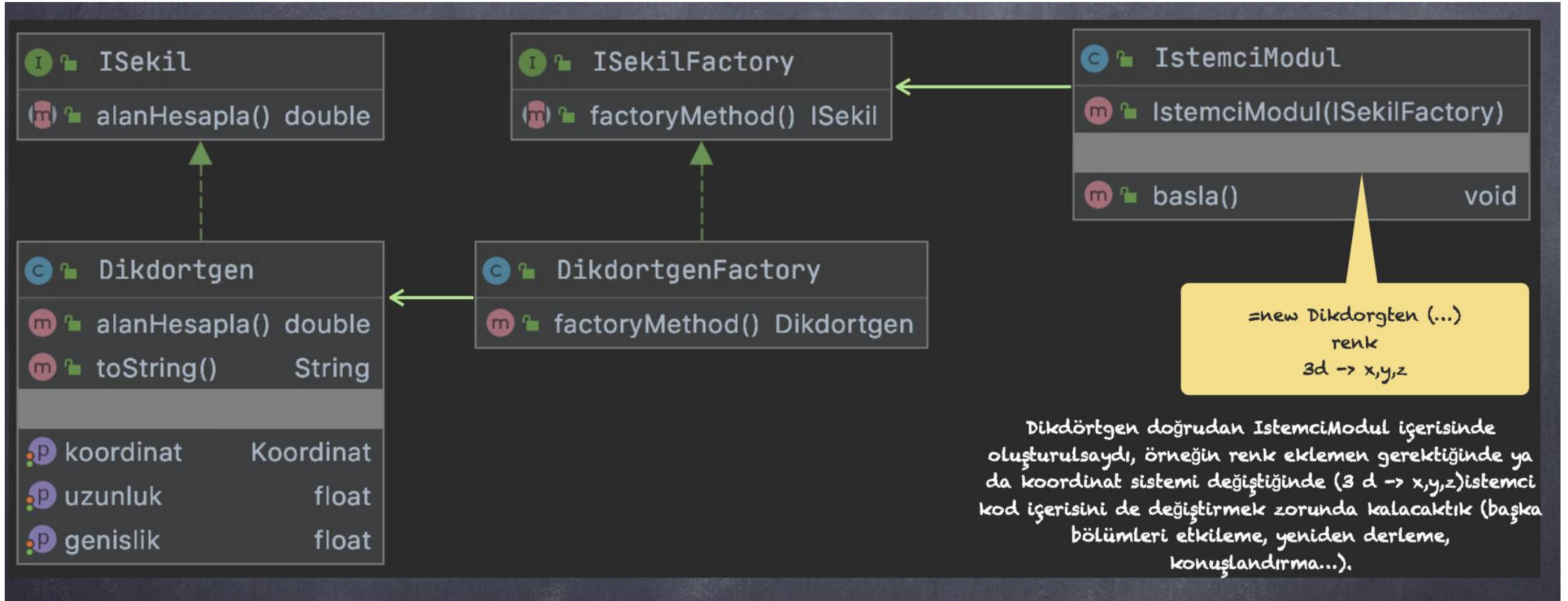
```
class HomeTheaterFacade {  
    private Lights _lights;  
    private Projector _projector;  
    private SoundSystem _sound;  
  
    public HomeTheaterFacade(Lights lights, Projector projector, SoundSystem sound)  
    {  
        _lights = lights;  
        _projector = projector;  
        _sound = sound;  
    }  
  
    public void WatchMovie() {  
        Console.WriteLine("Film başlıyor...");  
        _lights.Dim();  
        _projector.On();  
        _sound.SetVolume(10);  
    }  
}
```

```
var theater = new HomeTheaterFacade(new Lights(), new Projector(), new SoundSystem());  
theater.WatchMovie(); // Arka plandaki tüm karmaşa gizlendi.
```

Factory Tasarım Deseni

- Nesne oluşturma mantığını istemciden (client) gizleyen ve nesne üretimini tek bir noktaya odaklayan **yaratımsal (creational)** bir desendir.
Genellikle şu bileşenlerden oluşur:
- **Product (Ürün):** Oluşturulacak nesnelerin ortak arayüzü (Interface veya Abstract Class).
- **Concrete Product (Somut Ürün):** Arayüzü uygulayan gerçek sınıflar.
- **Creator (Factory):** Nesne üretimini gerçekleştiren metodun bulunduğu sınıf.

Factory Tasarım Deseni



Ne zaman kullanmalısın?

- Nesne türü önceden bilinmiyorsa:** Uygulamanın hangi nesneyle çalışacağına kullanıcı girişi veya bir yapılandırma dosyası karar veriyorsa.
- Nesne oluşturma süreci karmaşıksa:** Sadece new demek yetmiyor, nesneye bir sürü başlangıç ayarı yapılması gerekiyorsa bu karmaşıklığı factory içine hapsedersiniz.
- Kütüphane geliştiriyorsanız:** Kullanıcıya somut sınıfları göstermek istemiyor, sadece arayüz üzerinden işlem yapmasını istiyorsanız.

Dosya İşleyici

1. Ürün Arayüzü (Product)

```
public interface FileExporter {  
    String export(String content);  
}
```

2. Somut Ürünler (Concrete Products)

```
public class PdfExporter implements FileExporter {  
    @Override  
    public String export(String content)  
    { return "PDF dosyası oluşturuldu: " + content; }}
```

```
public class ExcelExporter implements FileExporter {  
    @Override  
    public String export(String content)  
    { return "Excel dosyası oluşturuldu: " + content; }}
```

3. Fabrika Sınıfı (Factory)

```
public class ExporterFactory {  
    public static FileExporter createExporter(String type)  
    { if (type.equalsIgnoreCase("PDF"))  
      { return new PdfExporter(); }  
      else if (type.equalsIgnoreCase("EXCEL"))  
      { return new ExcelExporter(); }  
      throw new IllegalArgumentException("Bilinmeyen dosya tipi!"); }}
```

4. Kullanım:

```
public class Main { public static void main(String[] args) { //  
    İstemci hangi sınıftan nesne üretildiğini bilmek zorunda değil  
    FileExporter exporter =  
    ExporterFactory.createExporter("PDF");  
    System.out.println(exporter.export("Yıllık Rapor")); }}
```

Strategy tasarım deseni

- **Strategy (Strateji) Tasarım Deseni**, bir işin yapılabileceği farklı yöntemleri (algoritmaları) kendi sınıflarına hapsedip, bu yöntemleri çalışma anında (runtime) birbirinin yerine kullanılabilir hale getiren **davranışsal (behavioral)** bir desendir.

Navigasyon Uygulaması

1. Strateji Arayüzü (Strategy)

Java

```
public interface RouteStrategy {  
    void buildRoute(String start, String end);  
}
```

2. Somut Stratejiler (Concrete Strategies)

Java

```
public class RoadStrategy implements RouteStrategy {  
    public void buildRoute(String start, String end) {  
        System.out.println("Araba için en hızlı yol hesaplanıyor...");  
    }  
}  
  
public class WalkingStrategy implements RouteStrategy {  
    public void buildRoute(String start, String end) {  
        System.out.println("Yaya için kestirme yollar hesaplanıyor...");  
    }  
}
```

3. Bağlam Sınıfı (Context)

Stratejiyi kullanan ve dışarıdan bir strateji alan sınıftır.

Java

```
public class Navigator {  
    private RouteStrategy strategy;  
  
    // Stratejiyi çalışma anında değiştirmemizi sağlar  
    public void setStrategy(RouteStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void executeRoute(String start, String end) {  
        strategy.buildRoute(start, end);  
    }  
}
```

4. Kullanım (Client)

Java

```
public class Main {  
    public static void main(String[] args) {  
        Navigator navigator = new Navigator();  
  
        // Kullanıcı "Araba" seçti  
        navigator.setStrategy(new RoadStrategy());  
        navigator.executeRoute("İstanbul", "Ankara");  
  
        // Kullanıcı vazgeçti, "Yürüyerek" gitmek istiyor  
        navigator.setStrategy(new WalkingStrategy());  
        navigator.executeRoute("Kadıköy", "Beşiktaş");  
    }  
}
```

Avantajları

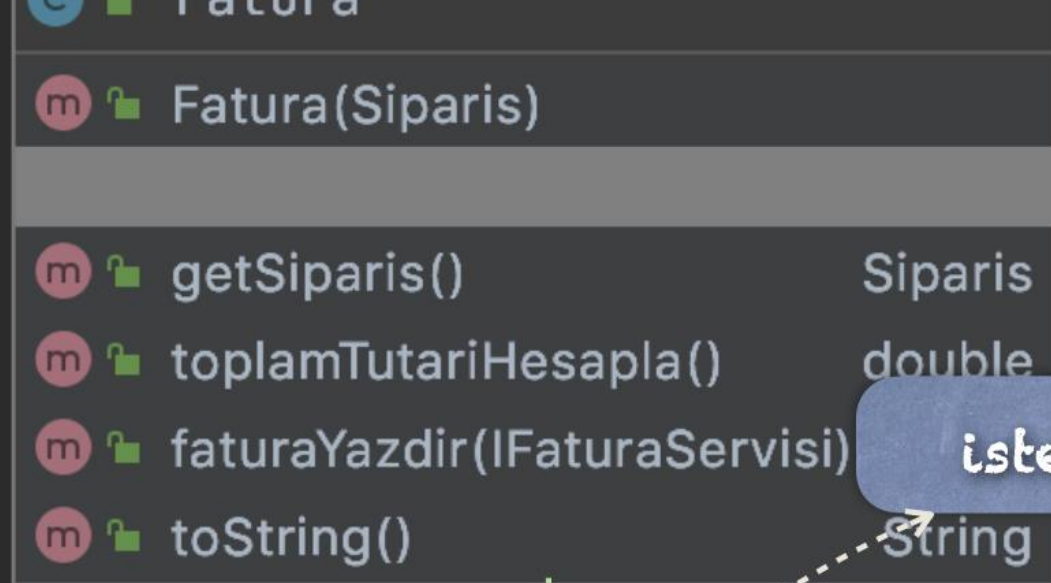
if-else Kalabalığından Kurtarır: Bir sürü `if(tip == A) ... else if(tip == B)` yazmak yerine strateji nesnesini değiştirirsiniz.

Genişletilebilirlik: Yeni bir yöntem eklemek için mevcut kodu bozmaz, sadece yeni bir sınıf yazarsınız.

Dezavantajları

Sınıf Sayısı Artar: Her yeni yöntem için yeni bir sınıf oluşturmanız gerekir.

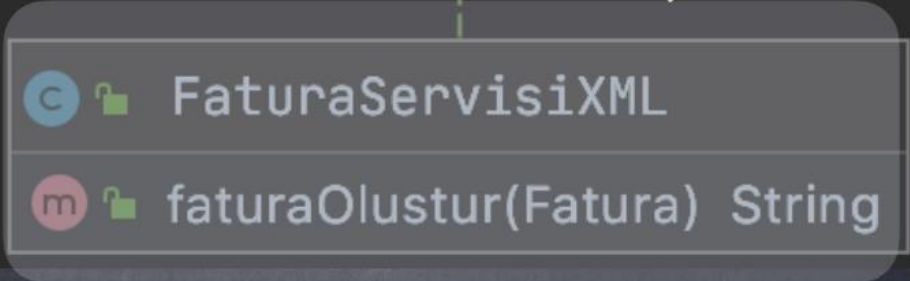
Karmaşıklık: İstemcinin hangi stratejinin ne işe yaradığını bilmesi gerekir.



istemci kod

farklı stratejiler

OCP
Open for extension,
Closed for
modification



Iterator Tasarım Deseni

- **Iterator (Yineleyici) Tasarım Deseni**, bir nesne grubunun (liste, ağaç, yığın vb.) iç yapısını bilmemize gerek kalmadan, elemanlarına sırayla erişmemizi sağlayan **davranışsal (behavioral)** bir desendir.

Neden Kullanmalıyız?

- **Sorumlulukların Ayrılması:** Koleksiyon sınıfı sadece veriyi saklamaya odaklanır, veriyi gezme mantığı Iterator sınıfına geçer.
- **Aynı Anda Farklı Gezinmeler:** Aynı koleksiyon üzerinde biri baştan sona, diğeri sondan başa giden iki farklı Iterator çalıştırabilirsiniz.
- **İç Yapıyı Gizleme:** Kullanıcı, verilerin bir ArrayList içinde mi yoksa bir HashMap içinde mi tutulduğunu bilmek zorunda kalmaz.

Veri toplulukları (collections) içerisinde elementler (nesneler) bulunur ve bu elementler çeşitli veri yapıları (dizi, bağlı liste, ağaç, graf vb.) kullanılarak bir arada tutulur.

Veri toplulukları içerisindeki her bir elemente erişilmesi gerekir. Bu işleme dolaşım (traversal) denir.

Veri toplulukları için kullanılan veri yapıları basit olduğunda (dizi, liste vb.) dolaşım için kullanılacak algoritma basit bir şekilde gerçekleştirilebilir.

Kullanılan veri yapıları karmaşıklıktıkça dolaşım algoritmaları zorlaşabilir ve zaman zaman farklı dolaşım algoritmalarına (ağaç veri yapısı için preorder, postorder, inorder gibi) ihtiyaç duyulabilir.

Iterator deseni, istemci modülün, veri topluluğu içerisinde kullanılacak dolaşım algoritmalarından etkilenmesini önlemek için, bu işlemi (sorumluluk) başka bir nesneye (iterator) yaptırır (SRP gereği).

istemci kod

Uygulama
main(String[]) void

IIterator
hasNext() boolean
next() Object

IContainer
getIterator() IIterator

PersonelCollection
personelListesi List<Personel>
getIterator() IIterator
getPersonelListesi() List<Personel>

PersonelIterator
index int
hasNext() boolean
next() Object

Personel
Personel(String, String, int, String)
toString() String
siciINO int
bolum String

Veri topluluğu
üzerindeki dolanım
algoritmalarından sorumlu
modül.

istemci kod

Uygulama
main(String[]) void

IIterator
hasNext() boolean
next() Object

IContainer
getIterator() IIterator

PersonelCollection
personelListesi List<Personel>
getIterator() IIterator
getPersonelListesi() List<Personel>

PersonelIterator
index int
hasNext() boolean
next() Object

Personel
Personel(String, String, int, String)
toString() String
siciINO int
bolum String

Veri topluluğu
üzerindeki dolanım
algoritmalarından sorumlu
modül.

istemci kod

Uygulama
main(String[]) void

IIterator
hasNext() boolean
next() Object

IContainer
getIterator() Iiterator

PersonelCollection
personellListesi List<Personel>
getIterator() Iiterator
getPersonellListesi() List<Personel>

PersonelIterator
index int
hasNext() boolean
next() Object

Personel
Personel(String, String, int, String)
toString() String
siciINO int
bolum String

Veri topluluğu
üzerindeki dolanım
algoritmalarından sorumlu
modül.

istemci kod

Uygulama
main(String[]) void

IIterator
hasNext() boolean
next() Object

IContainer
getIterator() Iiterator

PersonelCollection
personellListesi List<Personel>
getIterator() Iiterator
getPersonellListesi() List<Personel>

PersonelIterator
index int
hasNext() boolean
next() Object

Personel
Personel(String, String, int, String)
toString() String
siciINO int
bolum String

Veri topluluğu
üzerindeki dolanım
algoritmalarından sorumlu
modül.

Proxy Tasarım Deseni

- Bir nesneye erişimi kontrol etmek için kullanılan ve o nesnenin yerine geçen bir "temsilci" sınıf oluşturmaya dayanan **yapısal (structural)** bir desendir.
- Sınıfın fonk. Değiştirmeden davranışını değiştirmek istediğimizde kullanabiliriz.
- Sınıfı değiştirmek onu kullanan diğer sınıfların etkilenmesine neden olabilir.

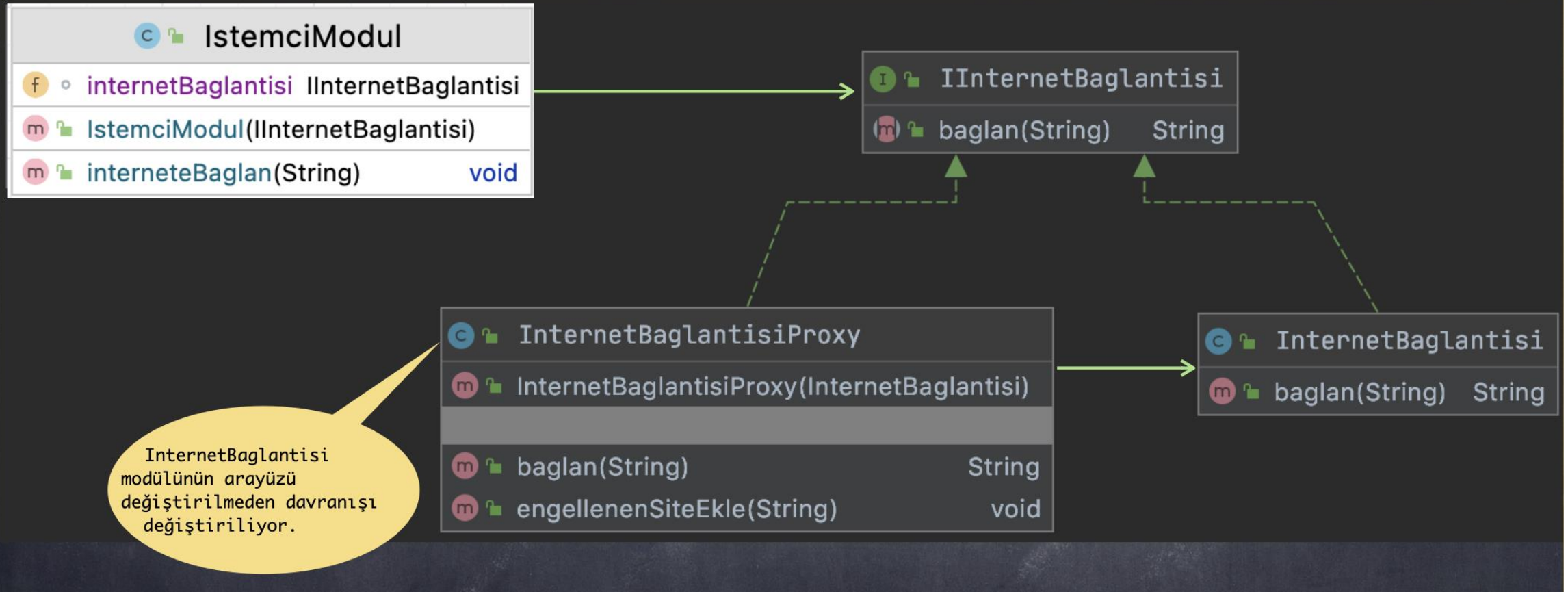
Örnek kullanım

- Bir nesneye erişimi kontrol etmek, güvenlik veya kimlik doğrulama sağlamak.
- Performansı artırmak için sık kullanılan veya maliyetli işlemleri önbelleğe alma.
- Tembel (lazy) başlatma işlemini uygulayarak gerçek nesneyi sadece gerektiğinde oluşturma.

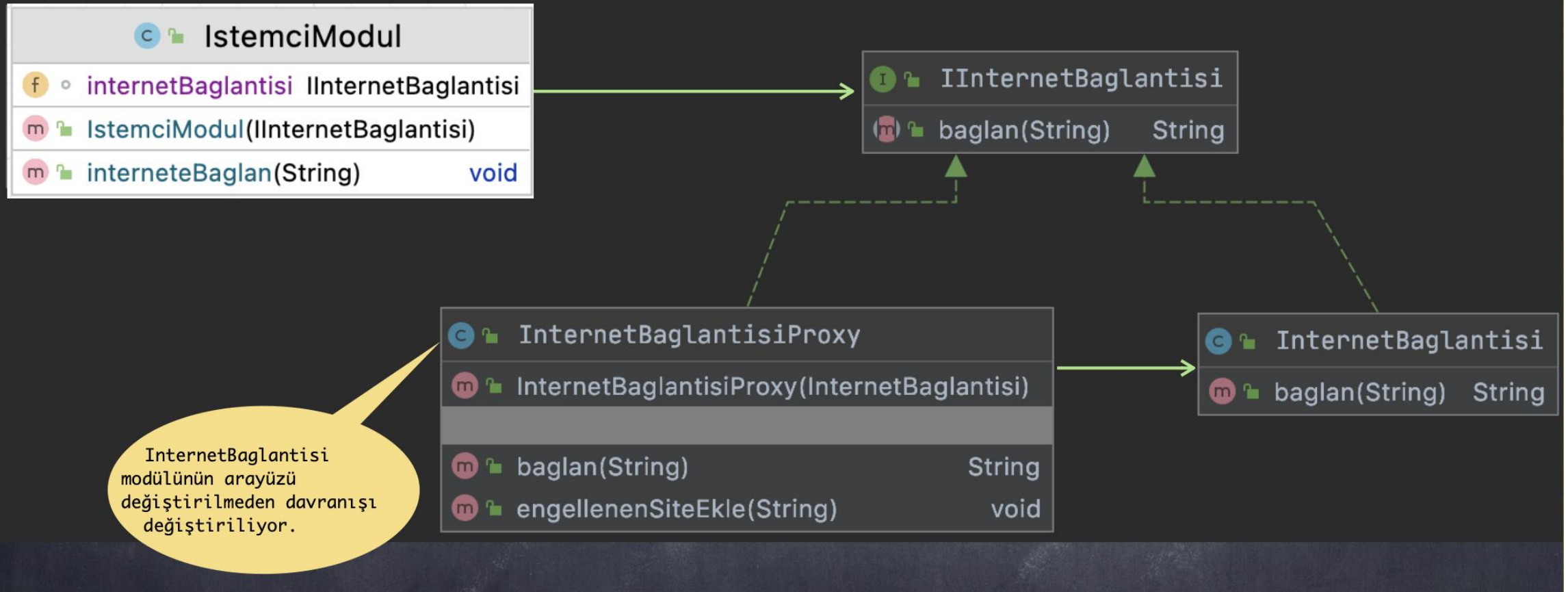
Neden ?

- Proxy deseni kullanım amacına göre 3 ana başlığa ayrılır:
- **Virtual Proxy (Sanal Vekil):** Ağır nesnelerin (büyük dosyalar, veritabanı bağlantıları) sadece ihtiyaç duyulduğunda oluşturulmasını sağlar.
- **Protection Proxy (Koruma Vekili):** Nesneye erişimi yetkilendirir. (Örn: "Admin değilse bu metodu çağırma").
- **Remote Proxy (Uzak Vekil):** Farklı bir sunucuda veya bellek alanında bulunan nesneye, sanki yanımızdaymış gibi erişmemizi sağlar.

Örnek



Örnek



Builder Tasarım Deseni

- **Builder (Kurucu) Tasarım Deseni**, karmaşık bir nesnenin adım adım inşa edilmesini sağlayan ve aynı inşa sürecinin farklı türde nesneler oluşturabilmesine imkan tanıyan **oluşturucu (creational)** bir desendir.

```

public class Computer {
    // Zorunlu alanlar
    private String HDD;
    private String RAM;

    // Opsiyonel alanlar
    private boolean isGraphicsCardEnabled;
    private boolean isBluetoothEnabled;

    // Getter metotları...

    // Constructor private olmalı ki sadece Builder erişebilsin
    private Computer(ComputerBuilder builder) {
        this.HDD = builder.HDD;
        this.RAM = builder.RAM;
        this.isGraphicsCardEnabled = builder.isGraphicsCardEnabled;
        this.isBluetoothEnabled = builder.isBluetoothEnabled;
    }

    // 2. Builder Sınıfı (Statik İç Sınıf)
    public static class ComputerBuilder {
        private String HDD;
        private String RAM;
        private boolean isGraphicsCardEnabled;
        private boolean isBluetoothEnabled;

        public ComputerBuilder(String hdd, String ram) {
            this.HDD = hdd;
            this.RAM = ram;
        }

        public ComputerBuilder setGraphicsCardEnabled(boolean isGraphicsCardEnabled) {
            this.isGraphicsCardEnabled = isGraphicsCardEnabled;
            return this; // "Method Chaining" için nesnenin kendisini döner
        }

        public ComputerBuilder setBluetoothEnabled(boolean isBluetoothEnabled) {
            this.isBluetoothEnabled = isBluetoothEnabled;
            return this;
        }

        public Computer build() {
            return new Computer(this);
        }
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        // Sadece ihtiyacımız olan özellikleri setliyoruz
        Computer comp = new Computer.ComputerBuilder("500 GB", "16 GB")
            .setGraphicsCardEnabled(true)
            .setBluetoothEnabled(false) // Bu opsiyonel, yazmasak da olurdu
            .build();
    }
}

```

Avantajları

Okunabilirlik: Hangi parametrenin ne olduğu metot isimlerinden anlaşılır.

Değişmezlik (Immutability): Nesne bir kez build() edildikten sonra setter'ları olmadığı için değiştirilemez.

Adım Adım İnşa: Nesne oluşturulurken araya mantıksal kontroller konulabilir.

Dezavantajları

Kod Fazlalığı: Her sınıf için bir de Builder sınıfı yazmak gerekir.

Karmaşıklık: Çok basit sınıflar için bu deseni kullanmak gereksiz "over-engineering" olabilir.

Abstract Factory

- **Abstract Factory (Soyut Fabrika) Tasarım Deseni**, birbiriyle ilişkili veya bağımlı nesne ailelerini, somut sınıflarını belirtmeden oluşturmak için bir arayüz sağlayan **oluşturucu (creational)** bir desendir.
- Factory Method tek bir ürün üretmeye odaklanırken, Abstract Factory bir **ürün ailesi** üretmekle ilgilenir.

Ne zaman tercih edilmeli?

- İstemci modül içerisinde, somut sınıflarını belirtmeden, birbirleriyle ilgili ya da birbirlerine bağlı nesne aileleri/ grupları oluşturmak için kullanılabilir.
- Örneğin; bir geliştirme ortamının görünümünü değiştirmek istediğimizde ya da bir yazılımın farklı platformlarda sorunsuz çalışabilmesini istediğimizde, bu deseni kullanabiliriz.

Günlük hayattan örnek

- Bir mobilya mağazasını düşünün. Müşterilerin tercih edebileceği farklı konseptler (stiller) vardır: **Modern**, **Viktoryen** ve **ArtDeco**.
- Bu konseptlerin her birinde aynı ürünler bulunur: **Sandalye**, **Masa** ve **Kanepe**.
- Modern Fabrikası =Modern Sandalye, Modern Masa üretir.
- Viktoryen Fabrikası = Viktoryen Sandalye, Viktoryen Masa üretir.
- Siz "Bana modern bir takım ver" dediğinizde, sistem size Viktoryen bir sandalye ile Modern bir masayı yanlışlıkla eşleştirmez. Çünkü fabrika, birbiriyle uyumlu olan "aile" ürünlerini üretir.

1. Ürün Arayüzleri

Java



```
public interface Button { void paint(); }  
public interface Checkbox { void paint(); }
```

2. Somut Ürünler (Windows Ailesi)

Java



```
public class WinButton implements Button {  
    public void paint() { System.out.println("Windows Düğmesi çizildi."); }  
}  
public class WinCheckbox implements Checkbox {  
    public void paint() { System.out.println("Windows Checkbox çizildi."); }  
}
```

3. Somut Ürünler (Mac Ailesi)

Java



```
public class MacButton implements Button {  
    public void paint() { System.out.println("Mac Düğmesi çizildi."); }  
}  
public class MacCheckbox implements Checkbox {  
    public void paint() { System.out.println("Mac Checkbox çizildi."); }  
}
```

4. Soyut Fabrika Arayüzü

Java



```
public interface GUIFactory {  
    Button createButton();  
    Checkbox createCheckbox();  
}
```

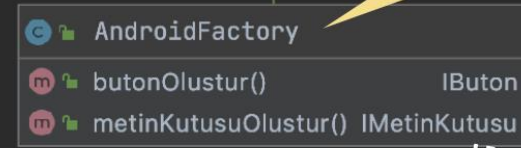
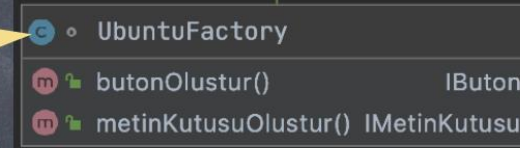
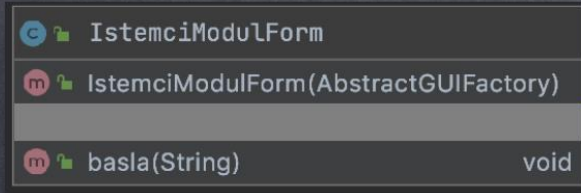
5. Somut Fabrikalar

Java



```
public class WinFactory implements GUIFactory {  
    public Button createButton() { return new WinButton(); }  
    public Checkbox createCheckbox() { return new WinCheckbox(); }  
}  
  
public class MacFactory implements GUIFactory {  
    public Button createButton() { return new MacButton(); }  
    public Checkbox createCheckbox() { return new MacCheckbox(); }  
}
```

```
public class Application {  
    private Button button;  
    private Checkbox checkbox;  
  
    // Uygulama sadece arayüzleri (interface) bilir, somut sınıfları değil!  
    public Application(GUIFactory factory) {  
        button = factory.createButton();  
        checkbox = factory.createCheckbox();  
    }  
  
    public void paint() {  
        button.paint();  
        checkbox.paint();  
    }  
}  
  
// Sistemin ayağa kalktığı yer (Configuration)  
public class Main {  
    public static void main(String[] args) {  
        Application app;  
        GUIFactory factory;  
  
        // Gerçek hayatta bu bilgi bir ayar dosyasından veya OS isminden gelir  
        String osName = System.getProperty("os.name").toLowerCase();  
  
        if (osName.contains("win")) {  
            factory = new WinFactory();  
        } else {  
            factory = new MacFactory();  
        }  
  
        // Uygulama hangi fabrikanın geldiğini bilmeden çalışır  
        app = new Application(factory);  
        app.paint();  
    }  
}
```



Android uyumlu nesne grupları (buton ve metin kutusu) oluşturuluyor.

Ubuntu uyumlu nesne grupları (buton ve metin kutusu) oluşturuluyor.

